

# Reviewer Pack — Arithmetic Divergence of Diophantine Sets Bounds Circuit Low...

Entry f10930da889d · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-28 21:33:40 UTC

## Arithmetic Divergence of Diophantine Sets Bounds Circuit Lower Bound for XOR Functions

**Entry ID:** f10930da889d **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-28 21:33:40 UTC

### 1. Conjecture

**Field A** (mathematical branch): Number Theory (Diophantine Equations) **Field B** (complexity object): Boolean Function Complexity

#### Statement:

[‘For every CNF formula  $F$  with  $n$  variables, the arithmetic divergence  $A(F)$  of the set of solutions to  $F$  is  $O(n^{1/4})$ . Moreover, for any circuit  $C$  computing XOR on  $n$  variables with depth  $D$  and size  $S$ ,  $A(C) = \Omega(S^{1/2}) + O(D^{1/4})$ .’, ‘For all CNF formulas  $F$  with  $n$  variables and circuits  $C$  computing XOR on  $n$  variables with depth  $D$  and size  $S$ , if  $A(F) \leq B$  for some constant  $B$ , then  $D \geq c * \log S$  for some absolute constant  $c$ .’, ‘Arithmetic divergence of the set of solutions to a CNF formula provides a lower bound on the size of any circuit computing XOR on the same number of variables.’]

#### Rationale (proposer’s reasoning):

[‘Diophantine equations have been used to study computational complexity, but their arithmetic aspects are less explored in this context.’, ‘Arithmetic divergence measures the average distance between solutions to a system of Diophantine equations and can be related to the complexity of circuits computing XOR functions.’, ‘This conjecture proposes a direct link between these two mathematical concepts, potentially revealing new insights into the nature of Boolean function complexity.’]

**Taxonomy category:** Arithmetic\_Diophantine\_Equations\_Boolean\_Circuit\_Complexity (status at proposal time: )

### 2. Pre-registration (Popper-style)

**Hash (SHA-256 prefix):** beefa0110446700a

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

#### Acceptance criterion:

A CNF formula  $F$  with  $n$  variables has arithmetic divergence  $A(F) \leq B$ , if for all generated XOR circuits  $C$  computing XOR on  $n$  variables with size  $S$  and depth  $D$ , the ratio of  $A(C)$  to  $\Omega(S^{1/2}) + O(D^{1/4})$  is less than 0.2.

### 3. Barrier filter (F1)

**Final:** PASS

| Barrier        | Verdict   | Confidence | LLM <sub>1</sub> | LLM <sub>2</sub> |
|----------------|-----------|------------|------------------|------------------|
| RELATIVIZATION | UNCERTAIN | 1.00       | HITS             | SAFE             |
| NATURAL_PROOFS | SAFE      | 0.50       | UNCERTAIN        | UNCERTAIN        |
| ALGEBRIZATION  | SAFE      | 0.50       | UNCERTAIN        | UNCERTAIN        |
| KARP_LIPTON    | SAFE      | 0.50       | UNCERTAIN        | UNCERTAIN        |

### 4. Novelty audit

**Verdict:** NOVEL against 9 hits across arXiv + Semantic Scholar + ECCC

**Search queries** (3): - arithmetic divergence Diophantine equations Boolean function complexity - circuit lower bound XOR functions Diophantine set bounds - CNF formula solutions arithmetic divergence lower bound circuits

**Top relevant hits considered:** - [http://arxiv.org/abs/2003.09703v1] Variance function of boolean additive convolution - [http://arxiv.org/abs/1609.06986v1] More on Diophantine sextuples - [http://arxiv.org/abs/1310.8140v2] Primes Solutions Of Linear Diophantine Equations - [http://arxiv.org/abs/2504.19966v3] Quantum circuit lower bounds in the magic hierarchy - [http://arxiv.org/abs/2304.02770v2] Tight Correlation Bounds for Circuits Between AC0 and TC0 - [http://arxiv.org/abs/0903.1996v1] Refinements of lower bounds for polygamma functions - [http://arxiv.org/abs/1406.3065v2] Lower Bounds for Tropical Circuits and Dynamic Programs - [http://arxiv.org/abs/2509.11322v1] Lower bounds for planar Arithmetic Circuits

### 5. Empirical test harness

**Multi-seed protocol:** 5 seeds (11, 23, 37, 53, 71)

**Sandbox:** isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

#### 5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = 10
    cnf = generate_cnf(n, seed)
    A_F = arithmetic_divergence(cnf)

    # Construct a corresponding XOR circuit with random size S and depth D (S, D ≤ 40)
    S = random.randint(5, 40)
    D = random.randint(5, 40)
    C = generate_xor_circuit(n, S, D, seed)
    A_C = arithmetic_divergence(C)

    # Compare A(C) with  $\Omega(S^{(1/2)}) + O(D^{(1/4)})$ 
    lower_bound = math.sqrt(S) + (D ** 0.25)
    ratio = A_C / lower_bound

    conjecture_holds = ratio < 0.2
    counterexample = "" if conjecture_holds else f"A(C)={A_C},  $\Omega(S^{(1/2)}) + O(D^{(1/4)})$ ={lower_bound}"
```

```

    return {
        "metric_name": "Arithmetic Divergence Ratio",
        "metric_value": ratio,
        "instances_tested": 1,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

def generate_cnf(n: int, seed: int) -> list:
    random.seed(seed)
    cnf = []
    for _ in range(n):
        clause = [random.choice([-1, 1]) for _ in range(n)]
        cnf.append(clause)
    return cnf

def arithmetic_divergence(formula: list) -> float:
    n = len(formula[0])
    solutions = set()

    # Find all solutions to the CNF formula
    def backtrack(assignment, clause_index):
        if clause_index == n:
            solutions.add(tuple(assignment))
            return
        for value in [-1, 1]:
            assignment[clause_index] = value
            if all(assignment[i] * clause[i] >= 0 for clause in formula):
                backtrack(assignment, clause_index + 1)

    backtrack([0] * n, 0)

    # Calculate arithmetic divergence
    total_distance = 0
    num_solutions = len(solutions)
    for i in range(n):
        distances = [abs(sum(solution[j] * formula[j][i] for j in range(n))) for solution in solutions]
        total_distance += sum(distances) / num_solutions

    return total_distance / n

def generate_xor_circuit(n: int, S: int, D: int, seed: int) -> list:
    random.seed(seed)
    circuit = []

    # Generate a random XOR circuit with depth D and size S
    def generate_layer(input_size):
        layer = [random.choice([-1, 1]) for _ in range(input_size)]
        return layer

    for _ in range(D - 1):
        input_size = len(circuit[-1]) if circuit else n
        circuit.append(generate_layer(input_size))

    # Add the final output layer

```

```

circuit.append(generate_layer(len(circuit[-1])))

return circuit

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if len(sys.argv) > 1 else list(range(2, 53))

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    metric_values = [r["metric_value"] for r in results]
    conjecture_holds_count = sum(r["conjecture_holds"] for r in results)

    mean = sum(metric_values) / len(metric_values)
    std = math.sqrt(sum((x - mean) ** 2 for x in metric_values) / len(metric_values))
    support_fraction = conjecture_holds_count / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean} std={std} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean} std={std} support_fraction={support_fraction}")
    else:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"Arithmetic Divergence Ratio\" first_failing_seed={first_failing_seed}")

```

## 6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

## 7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d8c21ee7.py", line 102, in <module>
    trial_result = run_trial(seed)
                   ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d8c21ee7.py", line 22, in run_trial
    A_F = arithmetic_divergence(cnf)
         ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d8c21ee7.py", line 67, in arithmetic_divergence
    backtrack([0] * n, 0)
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d8c21ee7.py", line 64, in backtrack
    if all(assignment[i] * clause[i] >= 0 for clause in formula):
        ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d8c21ee7.py", line 64, in <genexpr>
    if all(assignment[i] * clause[i] >= 0 for clause in formula):
        ^
NameError: cannot access free variable 'i' where it is not associated with a value in enclosing scope.

```

## 8. Critic adversarial review

**Critic verdict:** CHALLENGE

**Critic reasoning:**

skipped: test crashed before producing data

## 9. Final verdict & safety rail

**Verdict:** INCONCLUSIVE

**Reasoning:**

The test code crashed before producing data, which prevents us from verifying whether the conjecture's conditions are met. | next: Re-run the test with proper error handling to ensure it completes and produces results.

## 11. Audit log (LLM calls)

**Total LLM calls:** 10

| #  | Phase           | Provider      | Model            | Tokens in | out | Latency (ms) |
|----|-----------------|---------------|------------------|-----------|-----|--------------|
| 1  | propose         | ollama_remote | glm4:latest      | 0         | 0   | 16610        |
| 2  | propose         | ollama_remote | glm4:latest      | 0         | 0   | 11311        |
| 3  | preregistration | ollama_remote | glm4:latest      | 0         | 0   | 6325         |
| 4  | novelty         | ollama_remote | glm4:latest      | 0         | 0   | 4647         |
| 5  | novelty         | ollama_remote | glm4:latest      | 0         | 0   | 6126         |
| 6  | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 21154        |
| 7  | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 8378         |
| 8  | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 12533        |
| 9  | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 11943        |
| 10 | judge           | ollama_remote | glm4:latest      | 0         | 0   | 9966         |

**Totals:** 0 input tokens, 0 output tokens, ~\$0.0000 cost, 108992 ms total latency. Provider mix: {'ollama\_remote': 10}

*(full prompt+response transcripts available in research/audit/f10930da889d.jsonl)*

## 12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/f10930da889d.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

**Sandbox file:** research/pvsnp\_sandbox/test\_\*.py (per-cycle)

**Replay tarball:** research/replay/f10930da889d.tar.gz (if generated)